

Security Controls Mapping

Date updated: 2026-06-03

This document maps the backend/database implementation to the Computer Networks and Cyber-security part of the CS4455 Epic brief. It focuses on the implemented backend controls, the files that provide evidence, the tests that exercise them, and the residual risks that remain for the final submission.

Brief And Rubric Coverage

Brief requirement	Backend implementation	Evidence
Secure client/server connectivity using SSL/TLS	Public clients use <code>https://kfc.theburkenator.com</code> . TLS terminates at the public gateway, Nginx proxies to local Uvicorn, and FastAPI enforces HTTPS semantics through <code>ENFORCE_HTTPS</code> and trusted <code>X-Forwarded-Proto</code> handling.	<code>backend/app/main.py</code> , <code>backend/deploy/nginx/epic-messaging-api.conf</code> , <code>backend/scripts/check_tls_connection.py</code> , <code>docs/architecture/network_architecture.md</code> , <code>backend/tests/integration/test_security_headers.py</code> , <code>backend/tests/unit/test_tls_connection_probe.py</code>
Certificate authenticity and validity checks	The backend includes a TLS probe utility for validating the deployed hostname certificate and protocol. Client-side certificate validation is documented as a client requirement; backend deployment keeps the public API behind the TLS gateway.	<code>backend/scripts/check_tls_connection.py</code> , <code>docs/deployment/runbook.md</code> , <code>docs/architecture/network_architecture.md</code>
Server-side authentication	Registration stores Argon2id password hashes. Login returns generic failures. Access tokens are signed JWTs with required claims and <code>type=access</code> . Refresh tokens are high-entropy opaque values stored only as HMAC-SHA256 hashes. Refresh rotation revokes the old session under row lock.	<code>backend/app/services/password_service.py</code> , <code>backend/app/services/auth_service.py</code> , <code>backend/app/services/token_service.py</code> , <code>backend/app/repositories/refresh_session_repository.py</code> , <code>backend/tests/security/test_auth_security.py</code> , <code>backend/tests/unit/test_password_service.py</code> , <code>backend/tests/unit/test_token_service.py</code>

Brief requirement	Backend implementation	Evidence
Authorisation and broken access-control protection	Protected routes use <code>get_current_user()</code> . Message fetch/list/delete/revoke/forward and blockchain anchor lookup are scoped to the authenticated sender or recipient. Sender identity is taken from the JWT subject, not request input.	<code>backend/app/api/deps.py</code> , <code>backend/app/services/message_service.py</code> , <code>backend/app/repositories/message_repository.py</code> , <code>backend/app/services/blockchain_anchor_service.py</code> , <code>backend/tests/security/test_access_control_security.py</code> , <code>backend/tests/integration/test_message_routes.py</code> , <code>backend/tests/integration/test_blockchain_routes.py</code>
Input validation	Pydantic request models reject unknown fields and enforce UUID, positive integer, length, base64, and wire-payload structure constraints. Validation errors redact sensitive field names and submitted values.	<code>backend/app/schemas/common.py</code> , <code>backend/app/schemas/message.py</code> , <code>backend/app/schemas/device_key.py</code> , <code>backend/app/main.py</code> , <code>backend/tests/security/test_input_validation_security.py</code> , <code>backend/tests/unit/test_wire_payload_validation.py</code>
Injection resistance	Database access uses SQLAlchemy ORM expressions and <code>async AsyncSession</code> rather than string-built SQL. Login, user lookup, message, key, and anchor tests include malicious-looking inputs that stay data-only.	<code>backend/app/repositories/*.py</code> , <code>backend/tests/security/test_input_validation_security.py</code> , <code>backend/tests/security/test_auth_security.py</code>

Brief requirement	Backend implementation	Evidence
Cryptographic misuse controls in backend scope	Backend cryptography is limited to password hashing, token signing, refresh-token HMAC hashing, and Ethereum Keccak digest derivation for integrity anchors. Message payload encryption stays client-side; the backend stores opaque encrypted relay payloads only.	backend/app/services/password_service.py, backend/app/services/token_service.py, backend/app/core/blockchain_hashing.py, backend/app/models/message.py, backend/tests/security/test_sensitive_data_security.py, backend/tests/unit/test_blockchain_anchor_service.py
Sensitive data exposure controls	API responses omit password hashes and refresh-token hashes. User discovery hides email and public key material. Audit logging uses an allowlist and omits passwords, tokens, private keys, plaintext, and wire_payload_json.	backend/app/schemas/*.py, backend/app/services/audit_service.py, backend/tests/security/test_sensitive_data_security.py, backend/tests/integration/test_audit_logging.py
Security misconfiguration checks	Production settings reject wildcard CORS origins and placeholder/short JWT or refresh-token secrets. Real .env files are ignored. Security headers and HTTPS enforcement are implemented in middleware.	backend/app/core/config.py, backend/.env.example, .gitignore, backend/tests/integration/test_security_headers.py, backend/tests/security/test_sensitive_data_security.py
Vulnerable component checks	Bandit and pip-audit were run for the final DB-backed test pass. Both returned no findings on 2026-06-03.	docs/security/security_test_results.md
Vulnerability testing evidence	Security tests cover auth, access control, input validation, sensitive data, rate limiting, security headers, blockchain routes, and audit logging. The final DB-backed run passed.	backend/tests/security/*, backend/tests/integration/*, docs/security/security_test_results.md

Brief requirement	Backend implementation	Evidence
Network architecture documentation	Architecture docs record public TLS, gateway/Nginx/FastAPI/PostgreSQL boundaries, VM assumptions, ports, rate limiting, and blockchain worker separation.	docs/architecture/network_architecture.md, docs/architecture/backend_architecture.md, docs/deployment/runbook.md
Blockchain-backed fidelity proof	Message send and forward create pending Sepolia anchors using contract-compatible Keccak record IDs and digests over canonical encrypted message records. The production backend worker submits pending anchors and records transaction metadata.	backend/app/services/blockchain_anchor_service.py, backend/app/core/blockchain_hashing.py, backend/app/workers/blockchain_worker.py, backend/deploy/epic-messaging-blockchain-worker.service, backend/tests/integration/test_blockchain_routes.py

Implemented Backend Controls

Control area	Implemented control	Evidence	Residual risk and control
Password storage	Argon2id PHC hashes; plaintext passwords never stored.	password_service.py, test_password_service.py	Argon2id parameters require periodic review as hardware improves. Final submission records the current parameters and
Login and account enumeration	Duplicate registration and login failures return generic user-facing messages. Inactive users cannot authenticate.	auth_service.py, auth route/security tests	No MFA in backend prototype. Stolen valid credentials still authenticate until password reset/session revocation.
JWT access tokens	HS256 only, required sub, role, jti, iat, exp, and type claims; non-access tokens rejected by the Bearer dependency.	token_service.py, deps.py, token tests	Secret rotation is an operational process. Production settings reject weak or placeholder secrets.
Refresh sessions	Raw refresh tokens are high entropy and shown only once to the client. DB stores HMAC hashes. Refresh rotates and revokes the previous session under row lock.	auth_service.py, token_service.py, refresh_session_repository.py	No end-user session-management screen. Backend service contains logout_all() for server-side bulk revocation.
Rate limiting	Fixed-window IP limits for unauthenticated routes and user limits for authenticated operations. Nginx also provides public-edge request limiting.	deps.py, rate_limit.py, nginx/epic-messaging-api.conf, rate-limit tests	FastAPI limiter is in-memory and single-instance. Production VM keeps Nginx edge limits; larger deployments require a shared limiter.
Object-level access control	Message and anchor access uses current user ID and sender/recipient predicates. Revocation is sender-only. Delete is per-user visibility.	message_service.py, message_repository.py, blockchain_anchor_service.py	New routes must include the same object-level checks before exposing message or anchor data. Regression tests cover the current routes.

Control area	Implemented control	Evidence	Residual risk and control
Sender spoofing prevention	<code>sender_user_id</code> is not accepted in <code>message create schemas</code> ; service writes sender from <code>current_user.id</code> .	<code>schemas/message.py</code> , <code>message_service.py</code>	None for current message routes.
Public key relay	Backend stores public device keys and public one-time prekeys only. Private key fields are not present in <code>schemas/models</code> . Prekey bundle fetch consumes one one-time prekey when available.	<code>schemas/device_key.py</code> , <code>schemas/one_time_prekey.py</code> , key route tests	Server cannot prove ciphertext actually used the consumed prekey. That proof belongs to the client cryptography deliverable.
Encrypted message relay	Backend validates libsignal-style payload structure and stores <code>wire_payload_json</code> as opaque ciphertext.	<code>schemas/message.py</code> , <code>schemas/common.py</code> , <code>models/message.py</code> , <code>wire_payload</code> tests	Server sees metadata: sender, recipient, device IDs, timestamps, forwarding lineage, and visibility flags.
Input validation	Strict schemas reject unknown fields, invalid UUIDs, non-positive IDs, malformed base64, oversized payloads, and invalid wire JSON.	<code>schemas/common.py</code> , <code>schemas/*.py</code> , validation tests	Future schema changes require matching tests and documented limits.
Validation error sanitization	Request validation responses remove submitted input and redact sensitive field names.	<code>main.py</code> , <code>input-validation/sensitive-data</code> tests	Application logs still require careful operational handling outside request validation.
SQL injection prevention	Repository layer builds ORM queries; no request-controlled SQL string concatenation is used for backend queries.	<code>repositories/*.py</code> , Bandit result, injection-style tests	Future raw SQL requires explicit review and tests.

Control area	Implemented control	Evidence	Residual risk and control
Audit logging	Auth, key, message, and denied actions are recorded with allowlisted metadata.	<code>audit_service.py</code> , audit tests	No admin UI. Logs expose operational metadata and require access control at the server/DB level.
Security headers	Responses include no-sniff, frame-deny, no-referrer, no-store, permissions policy, and HSTS on HTTPS when enabled.	<code>main.py</code> , security header tests	HSTS depends on production HTTPS/proxy flags and public TLS behaviour.
CORS	Production rejects wildcard origins; credentials with wildcard origin are rejected.	<code>config.py</code> , security header/config tests	Allowed origins are deployment-specific and documented in the runbook.
Secrets handling	<code>.env.example</code> contains placeholders; real <code>.env</code> files are ignored; production rejects missing/short/placeholder secrets.	<code>backend/.env.example</code> , <code>.gitignore</code> , <code>config.py</code>	Host environment and systemd secret handling remain operational responsibilities.
Blockchain anchor creation	Send and forward create pending <code>blockchain_anchors</code> rows atomically with encrypted messages. Digests use Ethereum Keccak over canonical encrypted message metadata.	<code>message_service.py</code> , <code>blockchain_anchor_service.py</code> , <code>blockchain_hashing.py</code> , blockchain tests	Backend anchors prove integrity of stored encrypted metadata, not plaintext correctness. Plaintext proof is outside backend scope.
Blockchain submission	Production runs the separate backend blockchain worker with Sepolia RPC URL, worker wallet key, and contract address from environment. FastAPI request handlers do not hold wallet credentials or block on Sepolia.	<code>workers/blockchain_worker.py</code> , <code>scripts/run_blockchain_worker.py</code> , <code>deploy/epic-messaging-blockchain-worker.service</code> , <code>config.py</code>	Chain/RPC outage or worker failure leaves anchors pending until the worker resumes. Contract write restrictions are handled in the blockchain deliverable.

Control area	Implemented control	Evidence	Residual risk and control
Dependency security	ruff, compileall, bandit, pip-audit, and the full DB-backed pytest suite passed on 2026-06-03.	security_test_results.md	Dependency evidence is refreshed after package changes.

Residual Risks And Controls

Residual risk	Impact	Control in final submission
FastAPI rate limiting is in-memory	Multiple API processes do not share counters.	The VM deployment uses Nginx public-edge request limits. The limitation is documented for scale-out deployments.
No MFA	Password compromise gives an attacker an account path.	Password hashing, generic failures, rate limits, short-lived access tokens, and refresh rotation are implemented. MFA is outside the assessed backend prototype.
Metadata visible to backend	Honest-but-curious server sees sender/recipient IDs, device IDs, timestamps, forwarding lineage, and visibility state.	Backend never stores plaintext messages, private keys, or Signal session state. Threat model documents metadata exposure.
Public TLS terminates before FastAPI	Internal hop after gateway is HTTP.	Gateway terminates public TLS, Nginx and Uvicorn bind internally, FastAPI enforces forwarded HTTPS semantics in production, and the runbook documents exposed ports.
Blockchain worker depends on chain/RPC availability	Anchors stay pending during Sepolia, RPC, wallet, gas, or worker outages.	Production runs the worker as a separate service with environment configuration. Pending rows persist in PostgreSQL and are processed when the worker resumes.
Backend does not enforce smart-contract write policy	Contract-level authorization is not inside FastAPI/database code.	Backend worker submits through the configured wallet. Solidity write restrictions and deployment evidence are documented in the blockchain deliverable.
Compromised server can change public key material	Future users can receive attacker-controlled public keys from the compromised server.	Current backend reduces plaintext exposure. Stronger key transparency/safety-number controls are documented as outside backend scope.
Fully compromised client	Attacker reads plaintext before encryption or after decryption and can act with valid tokens.	Backend revokes refresh sessions when requested, but endpoint compromise is outside backend control.

